

Основные модели

Дисциплина: Имитационное моделирование

Верниковская Екатерина Андреевна

Содержание

1	Основные модели	5
1.1	Модель SIR	5
1.1.1	Базовая модель	6
1.1.2	Трёхпараметрическая модель SIR	8
1.2	Модель Лотки–Вольтерры	10
1.3	Программный код для модели SIR и графики	11
1.4	Программный код для модели Лотки–Вольтерры и графики	30
2	Выводы	55
3	Список литературы	56

Список иллюстраций

1.1	SIR	5
1.2	Ключевые параметры и их экологическая интерпретаци	11

Список таблиц

1 Основные модели

1.1 Модель SIR

Модель SIR - это фундаментальная математическая модель в эпидемиологии, которая описывает распространение инфекционного заболевания в закрытой популяции. Название происходит от трёх групп (компарментов), на которые делится популяция:

- S - Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться
- I - Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию
- R - Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи

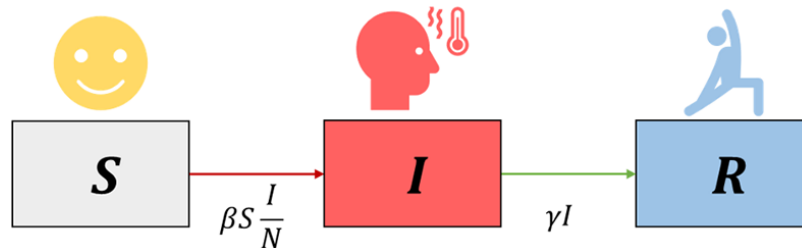


Рисунок 1.1: SIR

Ключевые допущения:

- Закрытая популяция: Общая численность населения $N = S + I + R$ постоянна (рождения, смерти от других причин и миграция не учитываются)
- Однородное смешение: Каждый восприимчивый имеет одинаковую вероятность встретиться с каждым заразным
- Постоянные параметры: Параметры заразности β и выздоровления γ не меняются со временем
- Отсутствие инкубационного периода и пожизненный иммунитет: После заражения человек сразу становится заразным, а после выздоровления приобретает стойкий иммунитет

1.1.1 Базовая модель

Динамика модели описывается тремя уравнениями, которые показывают, как меняется число людей в каждой группе во времени.

1. Уравнение для восприимчивых (S):

$$\frac{dS}{dt} = -\frac{\beta IS}{N}$$

- $\frac{dS}{dt}$ - скорость изменения числа восприимчивых
- β - коэффициент заражения (infection rate). Показывает, сколько в среднем человек заражает других в единицу времени, если все вокруг восприимчивы
- $-\frac{\beta IS}{N}$ - число новых случаев в единицу времени. Пропорционально и числу заразных (I), и доле восприимчивых (S/N). Знак минус показывает, что число восприимчивых уменьшается

2. Уравнение для заразных (I):

$$\frac{dI}{dt} = \frac{\beta IS}{N} - \gamma I$$

- $\frac{dI}{dt}$ - скорость изменения числа заразных
- $\frac{\beta IS}{N}$ - приток новых больных (т.е, кто перешёл из S)
- γ - коэффициент выздоровления (recovery rate). Обратная величина $1/\gamma$ - это средняя продолжительность болезни (в тех же единицах времени, что и t)
- γI - отток людей, которые выздоровели (перешли в R)

3. Уравнение для выздоровевших (R):

$$\frac{dR}{dt} = \gamma I$$

- $\frac{dR}{dt}$ - скорость изменения числа выздоровевших
- γI - поток людей, выздоравливающих из группы I

Ключевые параметры и показатели — это числовые характеристики модели (β , s (будет позже, в трёхпараметрической модели), γ) и производные от них величины (R_0 , R_e , порог коллективного иммунитета), которые определяют скорость распространения инфекции, силу эпидемии и позволяют прогнозировать её динамику, оценивать эффективность мер и принимать управленческие решения.

1. Базовое репродуктивное число R_0 : $R_0 = \frac{\beta}{\gamma}$ - среднее число людей, которых заражает один больной за все время своей болезни, в полностью восприимчивой популяции (когда $S \approx N$)
 - $R_0 > 1$: эпидемия будет расти (каждый больной заражает больше одного человека)
 - $R_0 < 1$: вспышка затухнет
2. Эффективное репродуктивное число R_e : $R_e = R_0 \frac{S}{N}$ - учитывает, что со временем восприимчивых становится меньше. Эпидемия начинает

сходится к нулю, когда R_e падает ниже 1 не из-за принятых мер, а из-за истощения пула восприимчивых

3. Порог коллективного иммунитета (Herd Immunity Threshold): $1 - \frac{1}{R_0}$ - это число, которое должна превысить доля иммунных (либо переболевших, либо вакцинированных), чтобы эпидемия не могла начаться

1.1.2 Трёхпараметрическая модель SIR

В классической двухпараметрической модели (β, γ) параметр β (коэффициент заражения) является составным. Он скрывает в себе два процесса:

- Контакт между людьми (поведенческий, управляемый фактор)
- Передачу инфекции при контакте (биологический фактор)

Трёхпараметрическая модель делает это разделение явным:

- c - среднее число контактов одного человека в единицу времени
- β - вероятность передачи инфекции при одном контакте между заразным и восприимчивым (величина от 0 до 1)
- γ - скорость выздоровления. А $1/\gamma$ - это средняя продолжительность заразного периода

Итоговый параметр силы заражения теперь выражается как произведение $c * \beta$

Математическая формулировка:

1. Уравнение для восприимчивых (S):

$$\frac{dS}{dt} = -\frac{\beta c I S}{N}$$

- $\frac{dS}{dt}$ - скорость изменения числа восприимчивых

- β - вероятность передачи инфекции при одном контакте
- c - среднее число контактов одного человека в день. Их произведение, $c \cdot \beta$, - это коэффициент заражения
- I - число заразных
- $\frac{S}{N}$ - доля восприимчивых в популяции
- $-\frac{\beta c I S}{N}$ - число новых заражений в день. Знак минус показывает, что число восприимчивых уменьшается

2. Уравнение для заразных (I):

$$\frac{dI}{dt} = \frac{\beta c I S}{N} - \gamma I$$

- $\frac{dI}{dt}$ - скорость изменения числа заразных
- $\frac{\beta c I S}{N}$ - приток новых больных (т.е, кто перешёл из S)
- γ - скорость выздоровления. Обратная величина $1/\gamma$ - это средняя продолжительность болезни в днях
- γI - отток людей, которые выздоровели (перешли в R)

3. Уравнение для выздоровевших (R):

$$\frac{dR}{dt} = \gamma I$$

- $\frac{dR}{dt}$ - скорость изменения числа выздоровевших
- γI - поток людей, выздоравливающих из группы I

Ключевые показатели:

1. Базовое репродуктивное число R_0 : $R_0 = \frac{c\beta}{\gamma}$

- $c\beta$ - это среднее число людей, которых заражает один больной за один день, если все вокруг восприимчивы
- $1/\gamma$ - это количество дней, которые человек в среднем заразен

$R_0 = (\text{число заражений в день}) \times (\text{число заразных дней}).$

Остальное без изменений

1.2 Модель Лотки–Вольтерры

Модель Лотки-Вольтерры - это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв.

Основные допущения модели:

- Закрытая система: нет миграции, популяции изолированы
- Неограниченные ресурсы для жертв: в отсутствие хищников жертвы растут экспоненциально
- Линейная функциональная реакция: вероятность встречи хищника и жертвы пропорциональна произведению их численностей
- Постоянные параметры: коэффициенты не меняются со временем
- Отсутствие внутривидовой конкуренции: нет конкуренции за ресурсы внутри вида
- Хищники питаются только жертвами: нет альтернативных источников пищи
- Отсутствие временных задержек: все процессы происходят мгновенно

Система состоит из двух дифференциальных уравнений.

1. Уравнение для жертв (x): $\frac{dx}{dt} = \alpha x - \beta xy$

где: $\alpha > 0$ - коэффициент естественного прироста жертв (рождаемость); $\beta > 0$ - коэффициент поедания жертв хищниками; xy - вероятность встречи хищника и жертвы (закон массового действия)

Численность жертв увеличивается за счёт размножения (αx) и уменьшается за счёт поедания хищниками (βxy)

2. Уравнение для хищников (y): $\frac{dy}{dt} = \delta xy - \gamma y$

где: $\delta > 0$ - коэффициент конверсии (эффективность превращения съеденных жертв в потомство хищников); $\gamma > 0$ - коэффициент естественной смертности хищников

Численность хищников увеличивается за счёт питания жертвами (δxy) и уменьшается за счёт естественной смертности (γy)

Параметр	Обозначение	Экологический смысл	Типичные значения
α	Альфа	Скорость размножения жертв (рождаемость)	0.1-1.0 день ⁻¹
β	Бета	Скорость поедания жертв хищниками	0.01-0.1 день ⁻¹
δ	Дельта	Эффективность конверсии пищи в потомство	0.01-0.1
γ	Гамма	Смертность хищников	0.1-0.5 день ⁻¹

Рисунок 1.2: Ключевые параметры и их экологическая интерпретации

1.3 Программный код для модели SIR и графики

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using JLD2
using StatsPlots
using LaTeXStrings # Для красивого отображения формул на графиках
using Plots
```

```

default(fmt = :png)
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end
    nothing
end
end

```

Activating project at `~/work/study/2025-2026/Имитационное моделирование/simulation modeling/labs/lab01/project`

```

└─ Warning: DrWatson could not find a project file by recursively checking given
  │ (given dir: /home/eavernikovskaya)
  └─ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

```

sir_ode! (generic function with 1 method)

Параметры модели

```

Δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0] # S, I, R
p = [0.05, 10.0, 0.25] # Δ, c, Δ

```

3-element Vector{Float64}:

```

0.05
10.0
0.25

```

Расчет базового репродуктивного числа

```

R0 = (p[2] * p[1]) / p[3] # R0 = (c * Δ) / Δ

```

2.0

Создание и решение задачи

```

prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = Δt)

```

retcode: Success

Interpolation: 3rd order Hermite

t: 15-element Vector{Float64}:

```

0.0
0.1
0.5331242537191838
1.3919008491040001
2.6102120755226386
4.1811820758323055

```

6.180123810566044
8.664951930903612
11.691425331813486
15.279370667110317
19.082963490744028
23.423425193442526
28.42949373098617
33.298779173639595
40.0

u: 15-element Vector{Vector{Float64}}:

[990.0, 10.0, 0.0]
[989.4990153200853, 10.247898072276143, 0.25308660763853125]
[987.1852964117763, 11.391111426852984, 1.4235921613706284]
[981.8312125227972, 14.026022042698546, 4.14276543450428]
[972.1392310589528, 18.757815561654187, 9.102953379393007]
[954.9905500491583, 27.007702518267607, 18.00174743257403]
[923.0706717929863, 41.92976779832721, 34.999560408686385]
[862.6705810342164, 68.49345419063417, 68.83596477514932]
[754.3022993851442, 109.74186185409279, 135.9558387607629]
[595.1014268847736, 150.41363264480682, 254.48494047041947]
[442.0141179101299, 154.80694988491865, 403.1789322049513]
[326.8951167493959, 119.06860394684855, 554.0362793037555]
[258.5085242188966, 70.11052194753069, 671.3809538335727]
[227.58338337085726, 37.328570631579986, 735.0880459975627]
[209.84121402014364, 14.47364496645583, 775.6851410134005]

Подготовка данных в DataFrame

```
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[:, :t] = sol_ode.t
df_ode[:, :N] = df_ode.S + df_ode.I + df_ode.R # Общая численность популяции
```

15-element Vector{Float64}:

```
1000.0
1000.0
 999.9999999999999
1000.0
1000.0
 999.9999999999999
1000.0
 999.9999999999999
1000.0
 999.9999999999999
 999.9999999999998
1000.0
1000.0
1000.0
1000.0
```

Вывод параметров модели

```
println("Параметры модели SIR:")
println("β (вероятность заражения) = ", p[1])
println("c (среднее число контактов) = ", p[2])
println("γ (скорость выздоровления) = ", p[3])
println("R0 = c * β / γ = ", round(R0, digits=3))
```

```
println("Средняя продолжительность болезни = ", round(1/p[3], digits=2), " дней")
println("Начальные условия: S0 = ", u0[1], ", I0 = ", u0[2], ", R0 = ", u0[3])
```

Параметры модели SIR:

β (вероятность заражения) = 0.05

c (среднее число контактов) = 10.0

γ (скорость выздоровления) = 0.25

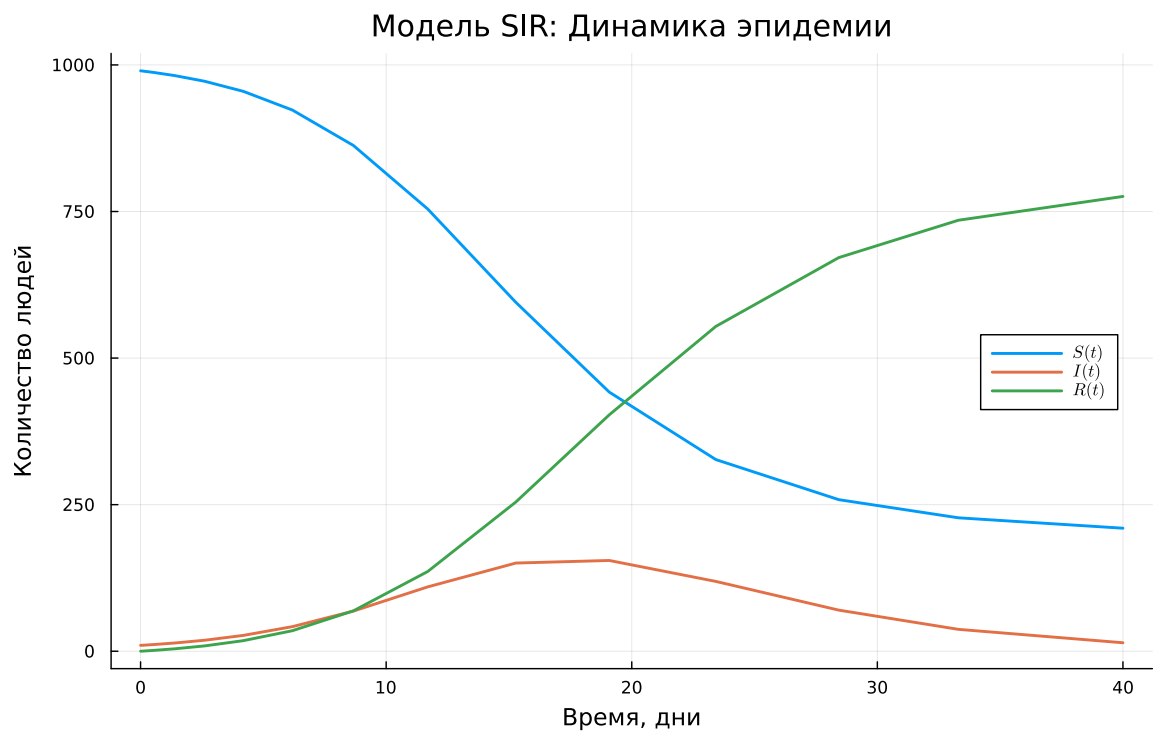
$R_0 = c * \beta / \gamma = 2.0$

Средняя продолжительность болезни = 4.0 дней

Начальные условия: $S_0 = 990.0$, $I_0 = 10.0$, $R_0 = 0.0$

1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп

```
plt1 = @df df_ode plot(:t,
    [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="Время, дни",
    ylabel="Количество людей",
    title="Модель SIR: Динамика эпидемии",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))
```

Добавление аннотаций с параметрами

```

annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
text("Параметры:\n $\beta$  =  $(p[1])$ \n $\gamma$  =  $(p[2])$ \n $\rho$  =  $(p[3])$ \n $R_0$  =  $(\text{round}(R_0, \text{digits}=2))$ "

```

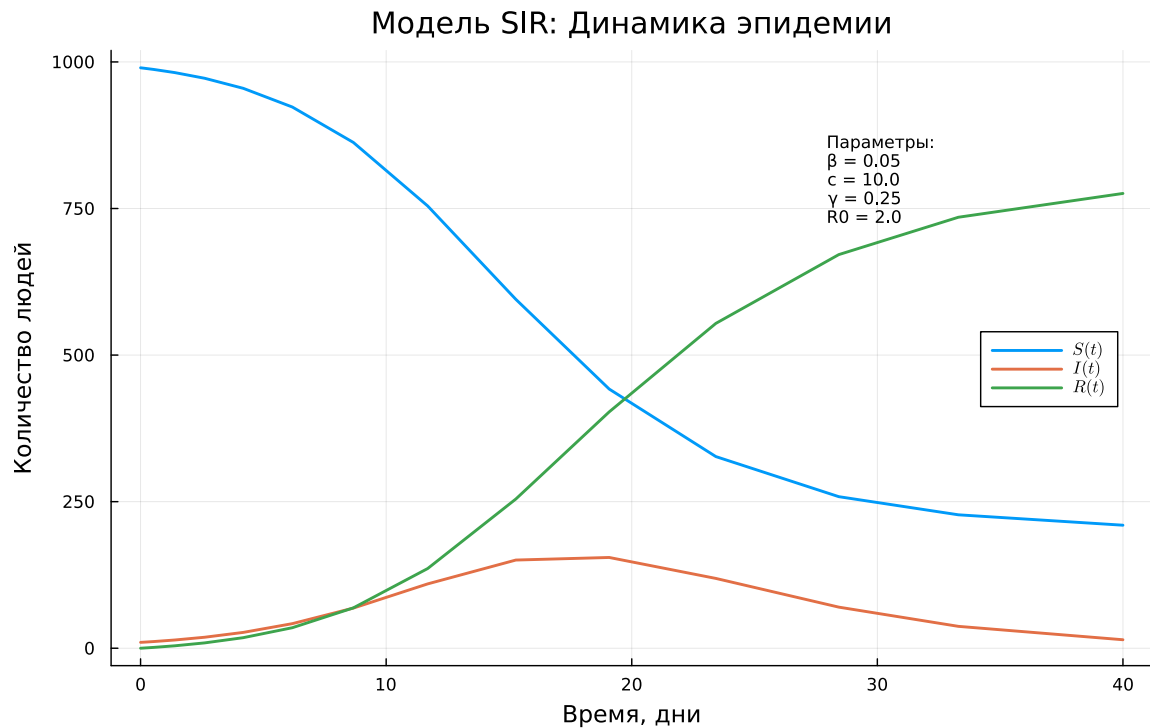
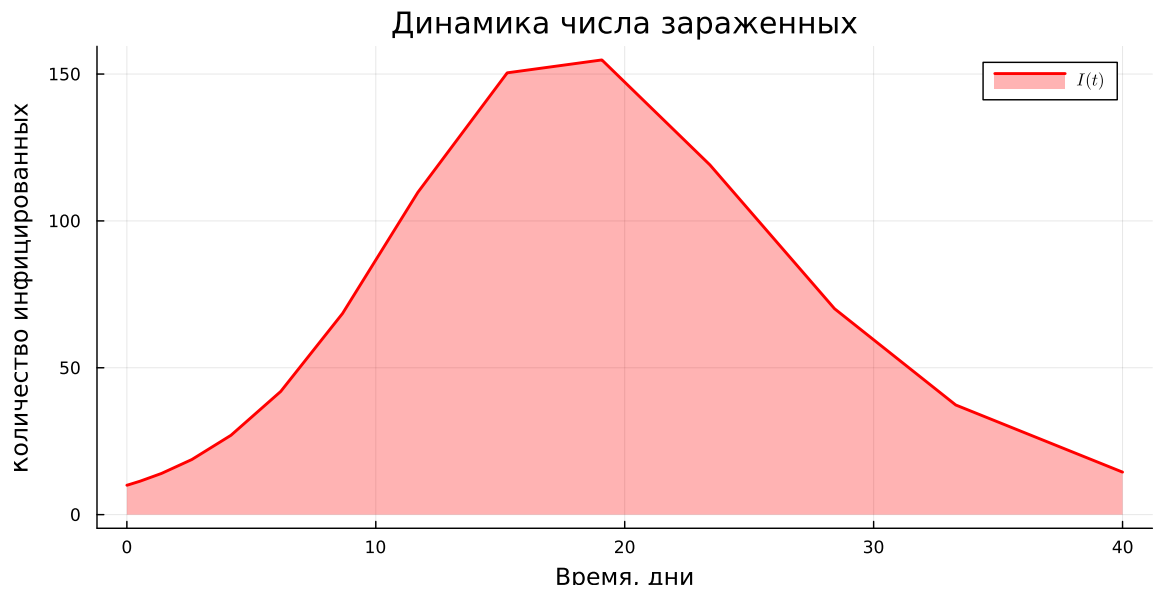


График только инфицированных (I)

```
plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="Количество инфицированных",
    title="Динамика числа зараженных",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))
```



Отметка пика эпидемии

```
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false, linewidth=1)
annotate!(plt2, peak_time, peak_value * 1.05,
text("Пик: $(round(peak_value, digits=1)) на $(round(peak_time, digits=1)) день", 8,
```

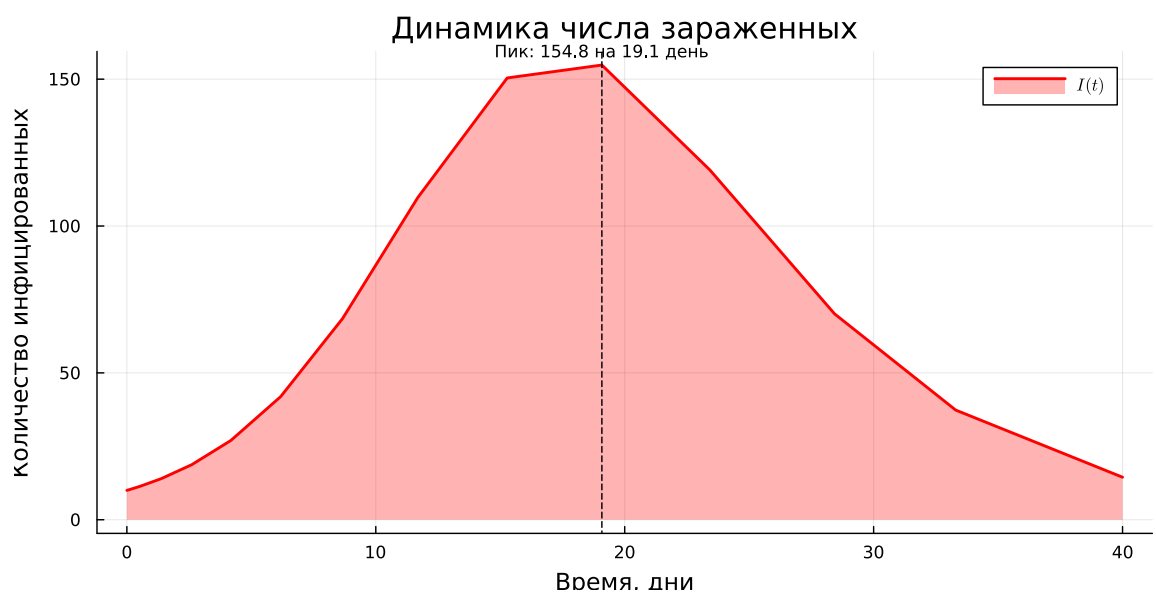


График в логарифмическом масштабе (для анализа экспоненциального роста)

```
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="Количество инфицированных (лог. масштаб)",
    title="Экспоненциальный рост (лог. шкала)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400))
```

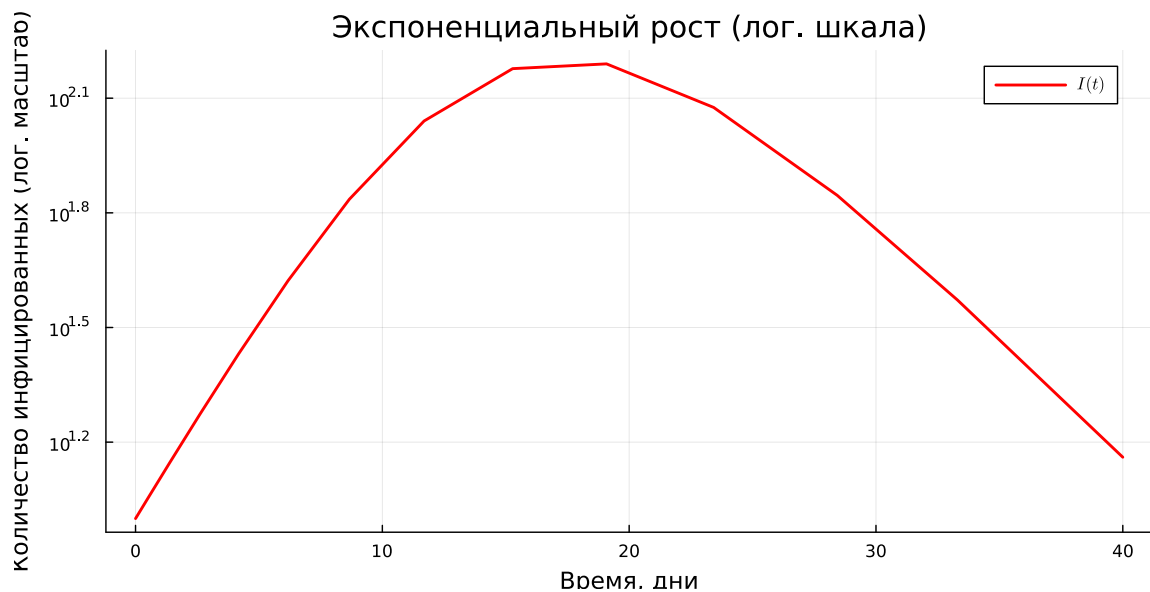


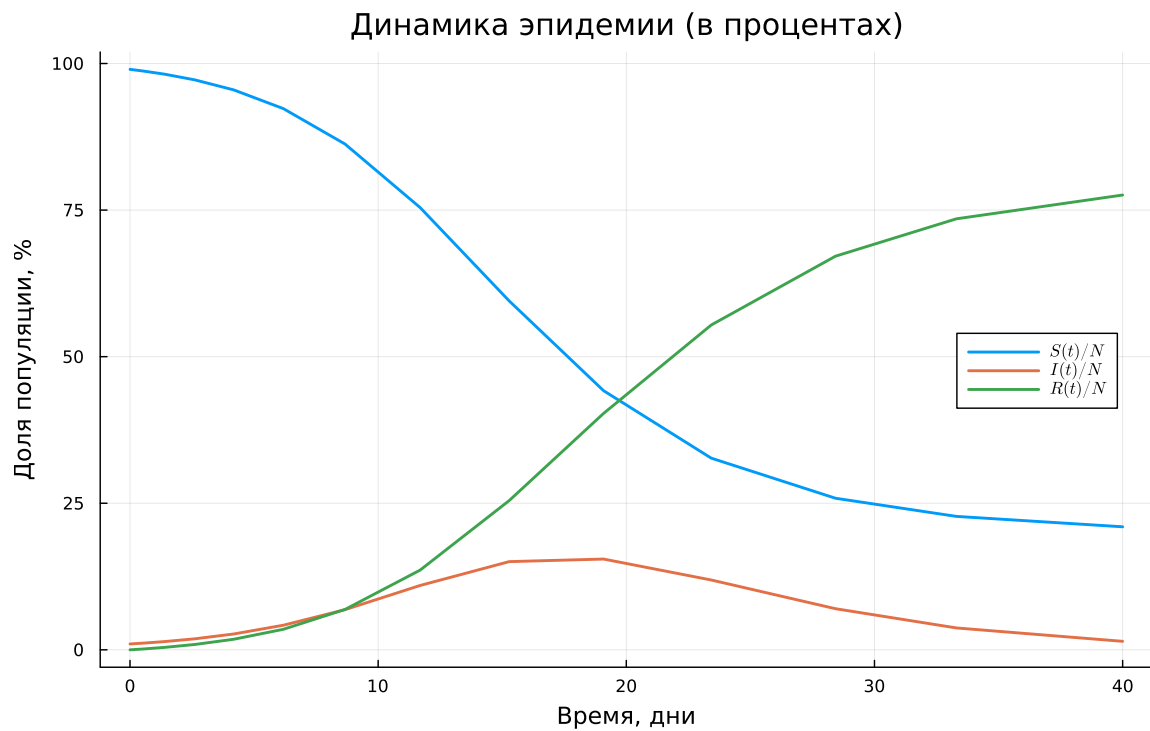
График долей населения (в процентах)

```
plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="Время, дни",
```

```

ylabel="Доля популяции, %",
title="Динамика эпидемии (в процентах)",
linewidth=2,
legend=:right,
grid=true,
size=(800, 500))

```

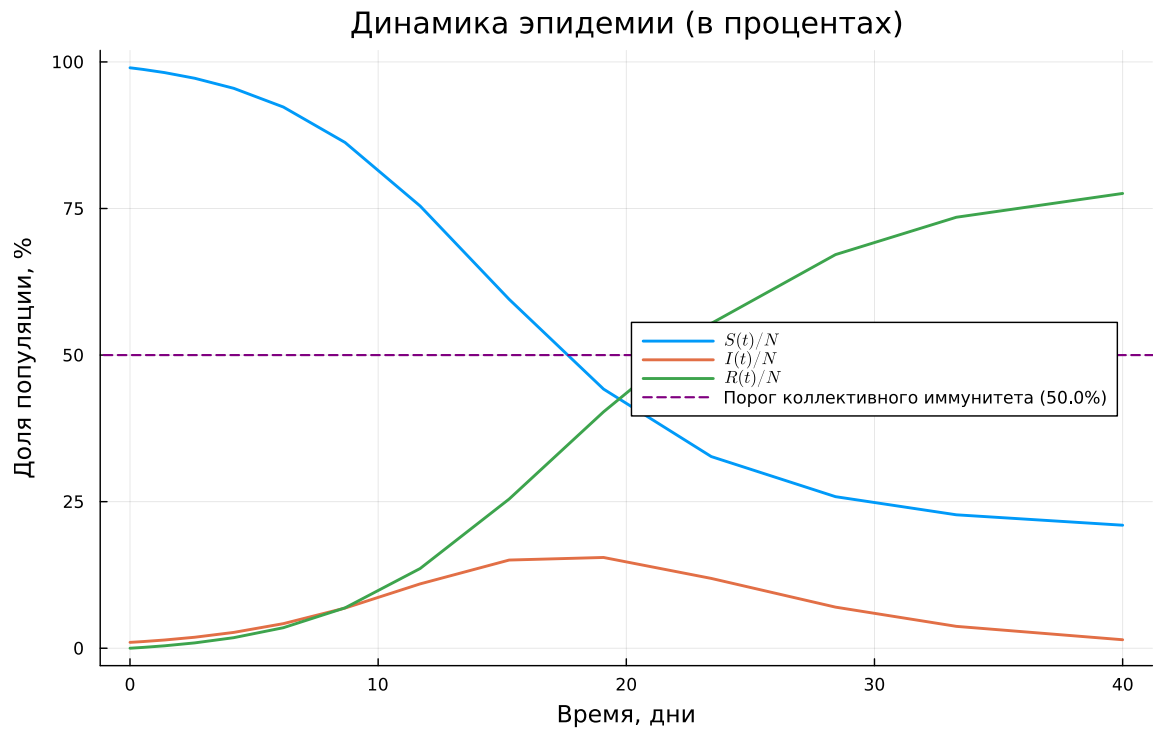


Горизонтальная линия для порога коллективного иммунитета

```

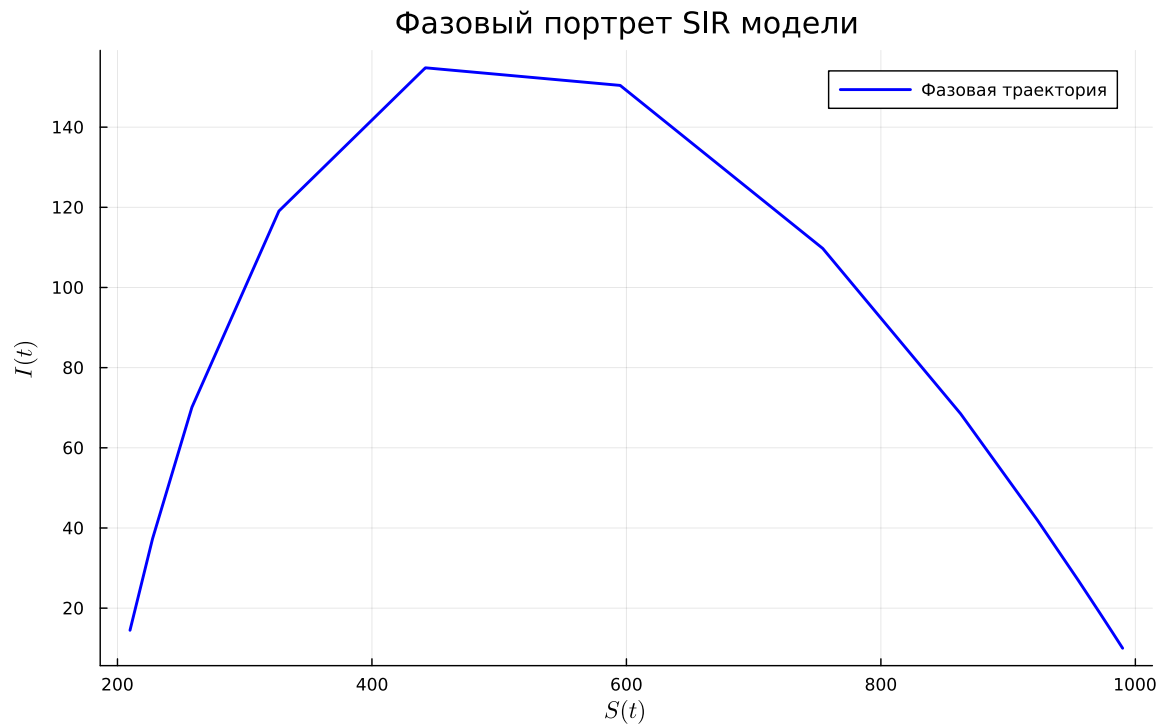
if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyle=:dash,
    label="Порог коллективного иммунитета ($(round(herd_immunity_threshold, digits=1))
end

```



Фазовый портрет (I vs S)

```
plt5 = plot(df_ode.S, df_ode.I,
            label="Фазовая траектория",
            xlabel=L"S(t)",
            ylabel=L"I(t)",
            title="Фазовый портрет SIR модели",
            color=:blue,
            linewidth=2,
            grid=true,
            size=(800, 500),
            legend=:topright)
```



Добавление стрелок направления

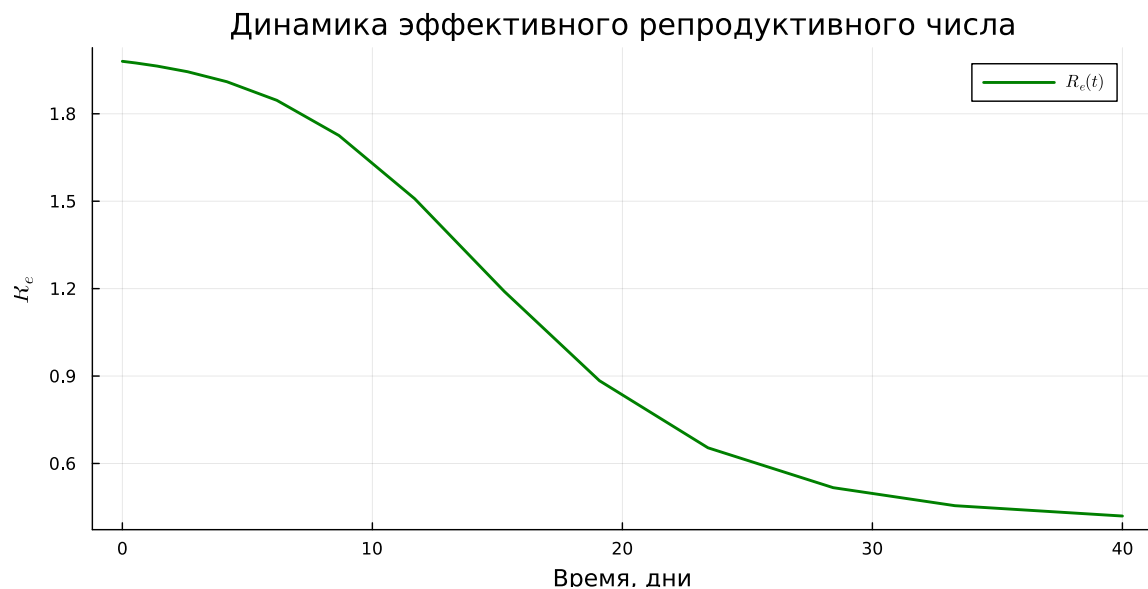
```
for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.I[i+1]],
           arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```

График R_e - эффективного репродуктивного числа

```
df_ode[:, :Re] = R0 .* df_ode.S ./ df_ode.N

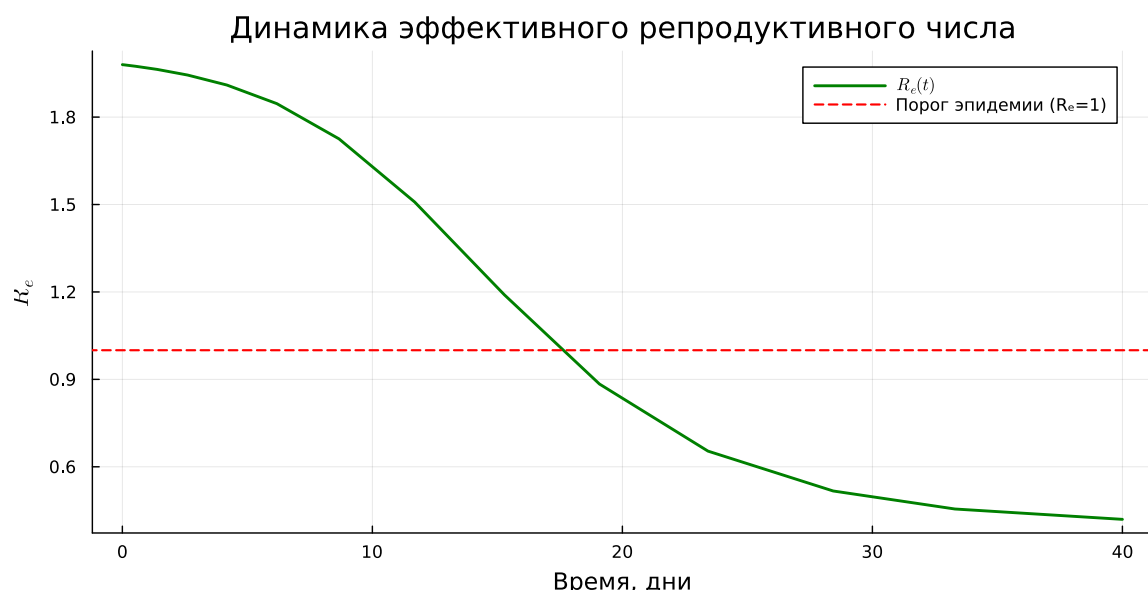
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
    xlabel="Время, дни",
    ylabel=L"R_e",
    title="Динамика эффективного репродуктивного числа",
    color=:green,
```

```
linewidth=2,
grid=true,
size=(800, 400))
```



Горизонтальная линия на уровне 1

```
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="Порог эпидемии ( $R_e=1$ )", linewidth=2)
```

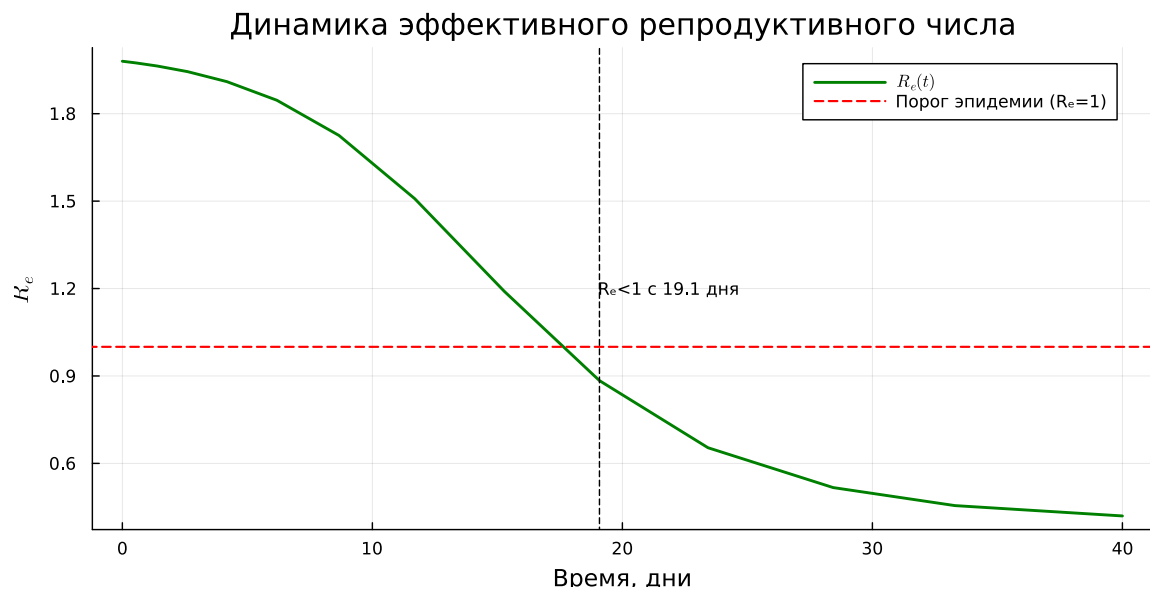


Отметка момента, когда R_e становится < 1


```

cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=false, linewidth=2)
    annotate!(plt6, cross_time, 1.2, text("R<1 с $(round(cross_time, digits=1)) дня")
end

```

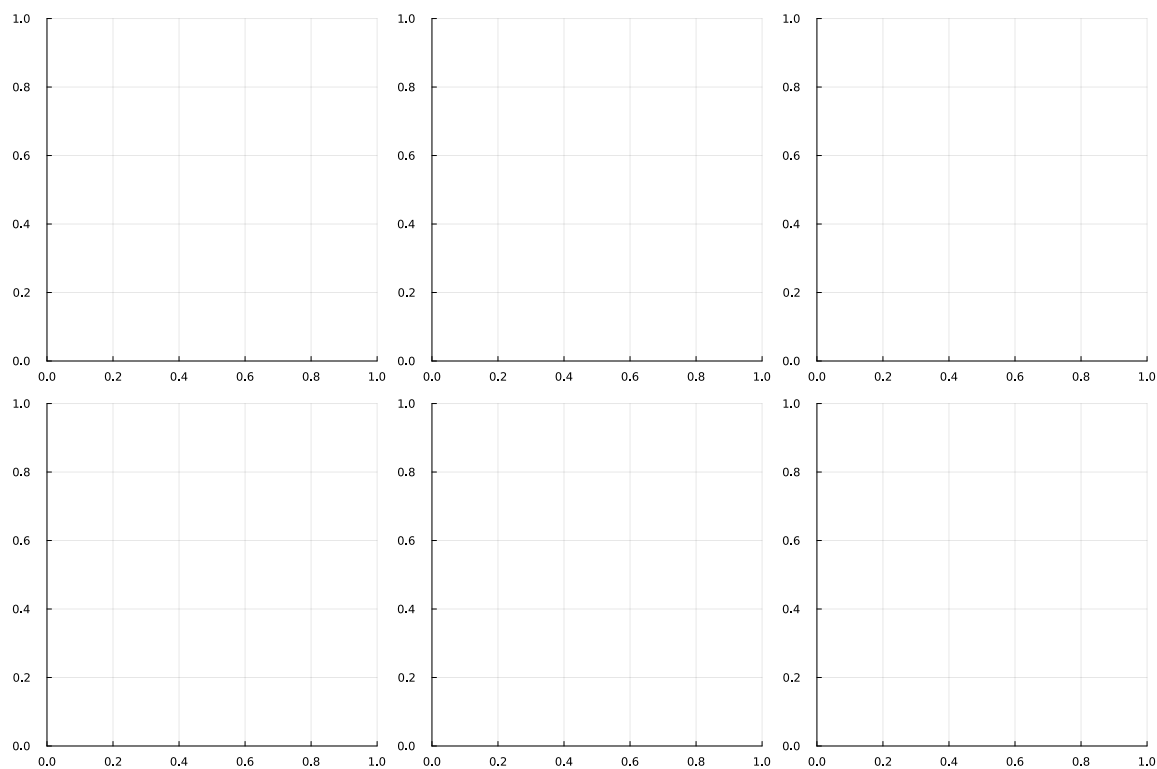


Компактный график всех кривых в одной панели

```

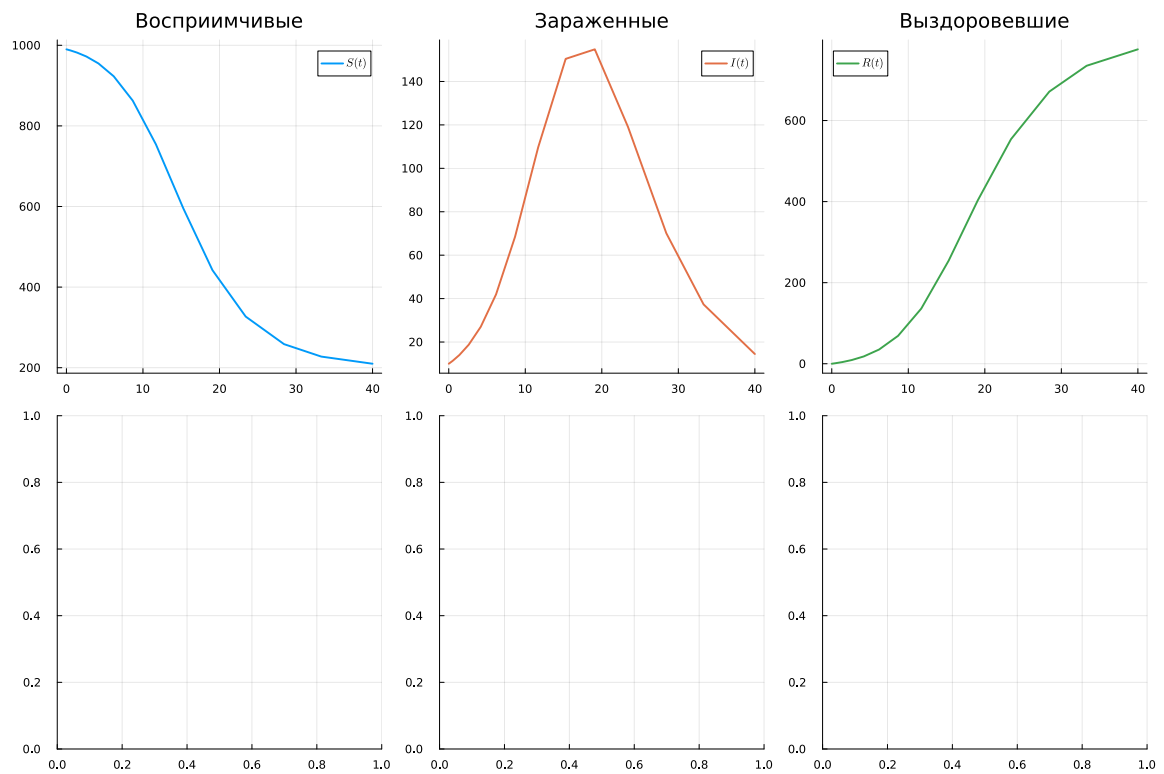
plt7 = plot(layout=(2, 3), size=(1200, 800))

```



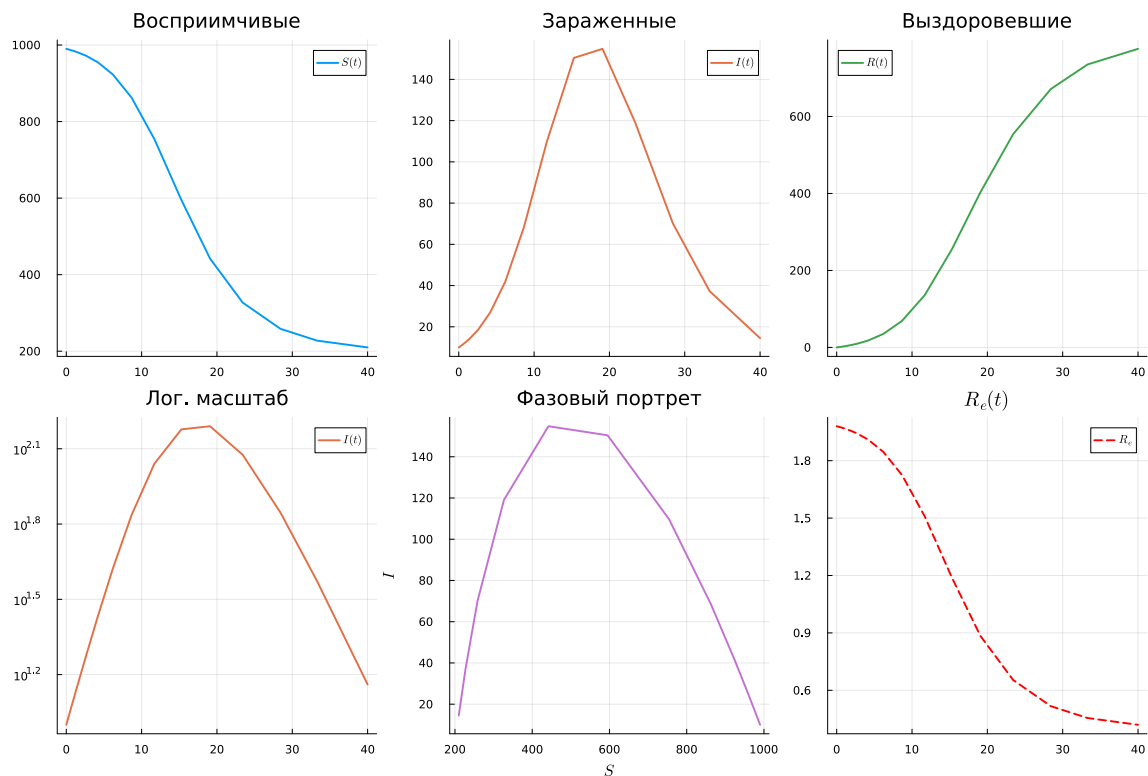
Верхний ряд

```
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidth=2, title="Воспри  
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth=2, title="Зараже  
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidth=2, title="Выздор
```



Нижний ряд

```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=:red, linewidth=2, yscale=:log10)
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=:blue, linewidth=2, title="Фазовый")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, linewidth=2, title=L"
```



Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

"/home/eavernikovskaya/work/study/2025-2026/Имитационное моделирование/simulation-modeling/labs/lab01/project/plots/sir_panel.png"

Бенчмарк для оценки производительности

```
println("\nБенчмарк решения:")
@benchmark solve(prob_ode, dt = Δt)
```

Бенчмарк решения:

BenchmarkTools.Trial: 10000 samples with 3 evaluations per sample.

Range (min ... max):	8.757 μ s ... 17.829 ms	GC (min ... max):	0.00% ... 99.79%
Time (median):	9.885 μ s	GC (median):	0.00%
Time (mean $\pm$ Δ):	13.669 μ s $\pm$ 178.202 μ s	GC (mean $\pm$ Δ):	13.02% $\pm$ 1.00%



Memory estimate: 17.39 KiB, allocs estimate: 333.

Дополнительный анализ

```
println("\n=== АНАЛИЗ РЕЗУЛЬТАТОВ ===")
println("Общая численность популяции (контроль): N = ", round(df_ode.N[1], digits=1))
println("Пиковое число зараженных: I_max = ", round(peak_value, digits=1))
println("Время достижения пика: t_peak = ", round(peak_time, digits=1), " дней")
println("Итоговое число переболевших: R(∞) = ", round(df_ode.R[end], digits=1))
println("Доля переболевших: ", round(df_ode.R[end]/df_ode.N[1]*100, digits=1), "%")

if R0 > 1
    println("\nТеоретический анализ:")
    println(" - Порог коллективного иммунитета: ", round((1-1/R0)*100, digits=1), "%")
```

```
println(" - Теоретический пик при  $S/N = 1/R_0 =$ ", round(1/R0, digits=3))  
end
```

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===

Общая численность популяции (контроль): $N = 1000.0$

Пиковое число зараженных: $I_{\max} = 154.8$

Время достижения пика: $t_{\text{peak}} = 19.1$ дней

Итоговое число переболевших: $R(\infty) = 775.7$

Доля переболевших: 77.6%

Теоретический анализ:

- Порог коллективного иммунитета: 50.0%
- Теоретический пик при $S/N = 1/R_0 = 0.5$

1.4 Программный код для модели Лотки–Вольтерры и графики

```
using Pkg  
Pkg.activate("../project")  
using DrWatson  
@quickactivate "project"  
using DifferentialEquations  
using DataFrames  
using StatsPlots  
using LaTeXStrings  
using Plots
```

```

using Statistics
using FFTW

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

```

Activating project at `~/work/study/2025-2026/Имитационное моделирование/simulation-modeling/labs/lab01/project`

└ Warning: DrWatson could not find a project file by recursively checking given
| (given dir: /home/eavernikovskaya)

└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

"/home/eavernikovskaya/work/study/2025-2026/Имитационное моделирование/simulation-modeling/labs/lab01/project/data/"

Описание модели Лотки-Вольтерры

```

"""
Модель Лотки-Вольтерры (хищник-жертва)
Система уравнений:
dx/dt =  $\alpha x - \beta xy$  # Изменение популяции жертв
dy/dt =  $\beta xy - \gamma y$  # Изменение популяции хищников
Где:
x - популяция жертв (например, зайцы)
y - популяция хищников (например, лисы)
 $\alpha$  - естественный прирост жертв (в отсутствие хищников)
 $\beta$  - коэффициент поедания жертв хищниками
 $\gamma$  - коэффициент прироста хищников за счет поедания жертв
 $\delta$  - естественная смертность хищников (в отсутствие жертв)

```

```

"""

function lotka_volterra!(du, u, p, t)
    x, y = u # x - жертвы, y - хищники
    a, b, c, d = p # параметры модели
    @inbounds begin
        du[1] = a*x - b*x*y
        du[2] = b*x*y - c*y
    end
    nothing
end

```

lotka_volterra! (generic function with 1 method)

Параметры модели и начальные условия Классические параметры из литературы

```

p_lv = [0.1, # a: скорость размножения жертв
        0.02, # b: скорость поедания жертв хищниками
        0.01, # c: коэффициент конверсии пищи (жертв) в хищников
        0.3] # d: смертность хищников

```

4-element Vector{Float64}:

```

0.1
0.02
0.01
0.3

```

Начальные условия: [жертвы, хищники]


```
u0_lv = [40.0, 9.0] # начальная популяция
```

2-element Vector{Float64}:

40.0

9.0

Временные параметры

```
tspan_lv = (0.0, 200.0) # длительность симуляции
```

```
dt_lv = 0.01 # шаг интегрирования
```

0.01

Создание и решение задачи

```
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
```

```
sol_lv = solve(prob_lv,
```

```
    Tsit5(), # Метод 5-го порядка
```

```
    reltol=1e-8, # Относительная точность
```

```
    abstol=1e-10, # Абсолютная точность
```

```
    saveat=0.1
```

```
)
```

retcode: Success

Interpolation: 1st order linear

t: 2001-element Vector{Float64}:

0.0

0.1

0.2

0.3

0.4

```

0.5
0.6
0.7
0.8
0.9
1.0
1.1
1.2
□
198.9
199.0
199.1
199.2
199.3
199.4
199.5
199.6
199.7
199.8
199.9
200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.677732019054936, 9.088990237143179]
 [39.35113789515877, 9.175882726251176]
 [39.02053587742845, 9.260562174604983]
 [38.68624820359512, 9.34291631164816]
 [38.34860018549613, 9.422836266192155]
 [38.00791932148964, 9.500216918207974]

```

[37.66453436579186, 9.574957251506675]
[37.31877446910067, 9.646960666011953]
[36.97096821728331, 9.716135308386768]
[36.6214428710377, 9.782394322077632]
[36.270523370595285, 9.845656152960181]
[35.91853162632679, 9.905844752781562]

□

[17.889675635852143, 5.712401889017661]
[17.865436241348913, 5.643570487712612]
[17.843673451560985, 5.575440227126896]
[17.8243531444655, 5.508019317739074]
[17.807442038019037, 5.441315149286253]
[17.792907688913118, 5.375334294518775]
[17.780718472979853, 5.310082579284777]
[17.77084358542411, 5.245565092116126]
[17.763253011515747, 5.18178624759267]
[17.757917518947043, 5.118749806042619]
[17.75480865783271, 5.05645887354262]
[17.753898747815317, 4.994915937021223]

Подготовка данных

```
df_lv = DataFrame()  
df_lv[:, :t] = sol_lv.t  
df_lv[:, :prey] = [u[1] for u in sol_lv.u] # жертвы  
df_lv[:, :predator] = [u[2] for u in sol_lv.u] # хищники
```

2001-element Vector{Float64}:

9.0

9.088990237143179

9.175882726251176
9.260562174604983
9.34291631164816
9.422836266192155
9.500216918207974
9.574957251506675
9.646960666011953
9.716135308386768
9.782394322077632
9.845656152960181
9.905844752781562

□

5.712401889017661
5.643570487712612
5.575440227126896
5.508019317739074
5.441315149286253
5.375334294518775
5.310082579284777
5.245565092116126
5.18178624759267
5.118749806042619
5.05645887354262
4.994915937021223

Расчет производных для анализа

```
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .* df_lv.predator  
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .- p_lv[4] .* df_lv
```

2001-element Vector{Float64}:

0.90000000000000004
0.8796081183887825
0.8580494468298019
0.8353523334118109
0.8115489002839338
0.7866749259983155
0.7607697061766037
0.7338758890516086
0.7060392942682667
0.677308704295883
0.6477356514560064
0.6173741700598212
0.5862805545482095
□
-0.6917903977437483
-0.6848226590959028
-0.6777687205225754
-0.6706369808625302
-0.6634355034707774
-0.6561720193614177
-0.6488539396800597
-0.6414883599432729
-0.634082072601987
-0.6266415732544758
-0.6191730642032904
-0.6116824631091282

Вывод информации о модели

```
println("="^60)
println("Модель Лотки-Вольтерры (хищник-жертва)")
println("="^60)
println("\nПараметры модели:")
println("□ (скорость размножения жертв) = ", p_lv[1])
println("□ (скорость поедания жертв) = ", p_lv[2])
println("□ (коэффициент конверсии) = ", p_lv[3])
println("□ (смертность хищников) = ", p_lv[4])
println("\nНачальные условия:")
println("Жертвы (x0) = ", u0_lv[1])
println("Хищники (y0) = ", u0_lv[2])
```

```
=====
Модель Лотки-Вольтерры (хищник-жертва)
=====
```

Параметры модели:

□ (скорость размножения жертв) = 0.1
 □ (скорость поедания жертв) = 0.02
 □ (коэффициент конверсии) = 0.01
 □ (смертность хищников) = 0.3

Начальные условия:

Жертвы (x0) = 40.0
 Хищники (y0) = 9.0

Стационарные точки (нулевые изоклины)

```

x_star = p_lv[4] / p_lv[3] # стационарная точка для жертв
y_star = p_lv[1] / p_lv[2] # стационарная точка для хищников
println("\nСтационарные точки (положения равновесия):")
println("x* = □/□ = ", round(x_star, digits=3))
println("y* = □/□ = ", round(y_star, digits=3))

```

Стационарные точки (положения равновесия):

$x^* = \square/\square = 30.0$

$y^* = \square/\square = 5.0$

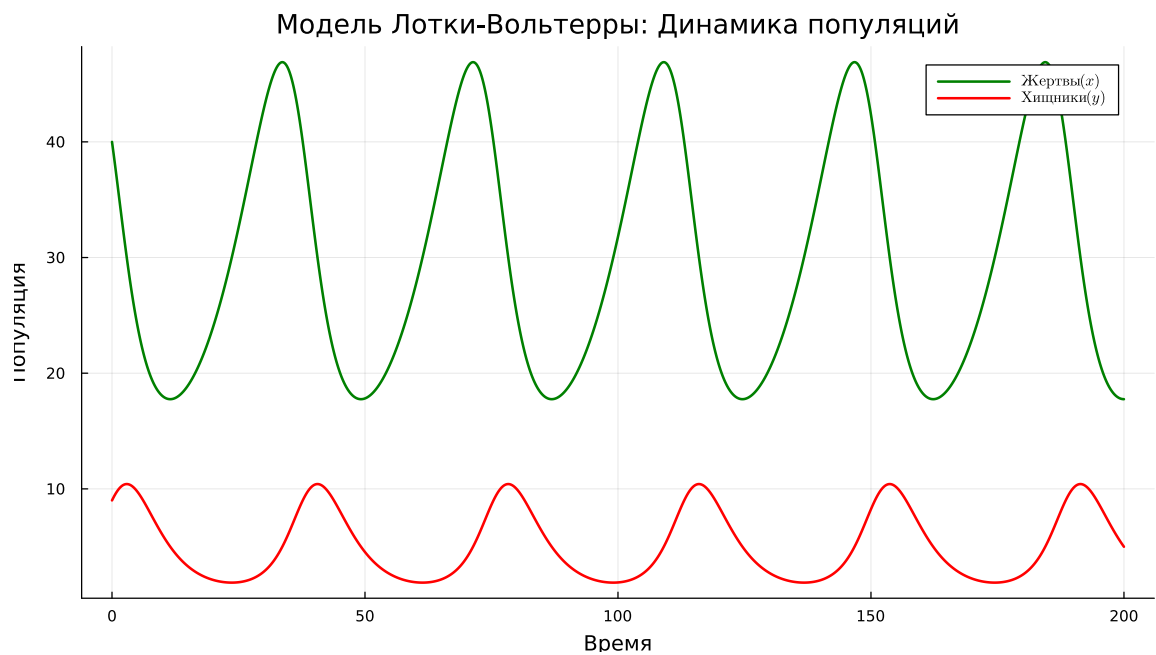
Построение графиков

График 1: Динамика популяций во времени

```

plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
            label=[L"Жертвы (x)" L"Хищники (y)"],
            xlabel="Время",
            ylabel="Популяция",
            title="Модель Лотки-Вольтерры: Динамика популяций",
            linewidth=2,
            legend=:topright,
            grid=true,
            size=(900, 500),
            color=[:green :red])

```



Добавление стационарных уровней

```
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5, label="x* (равновесие жертв)"
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, label="y* (равновесие хищников)"
```

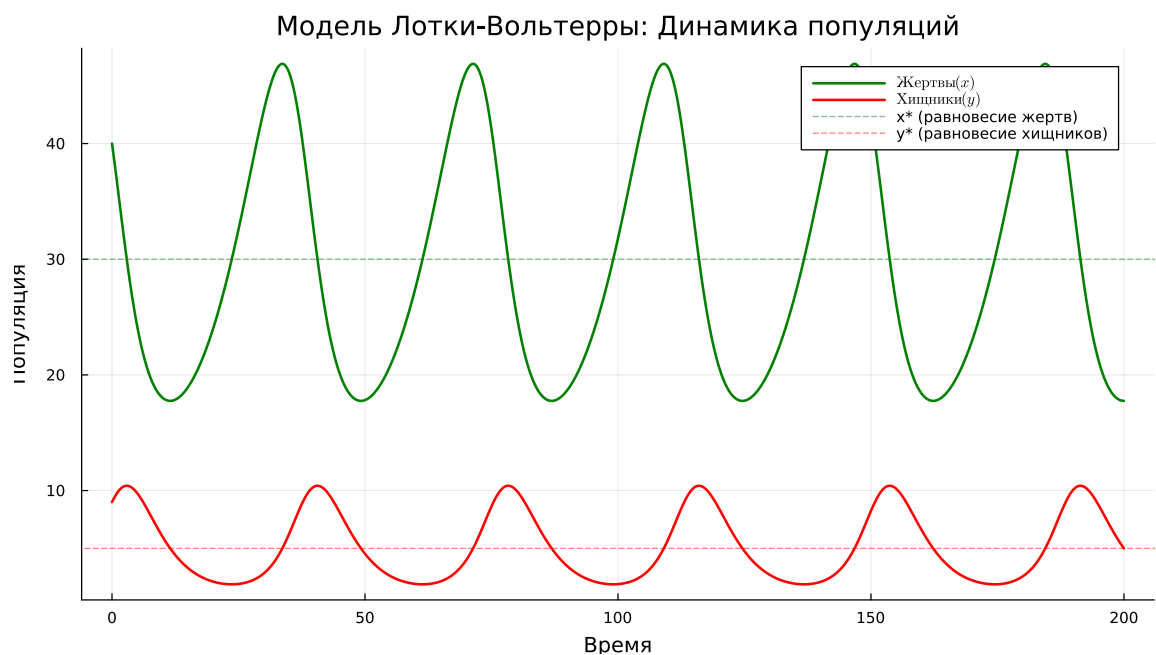
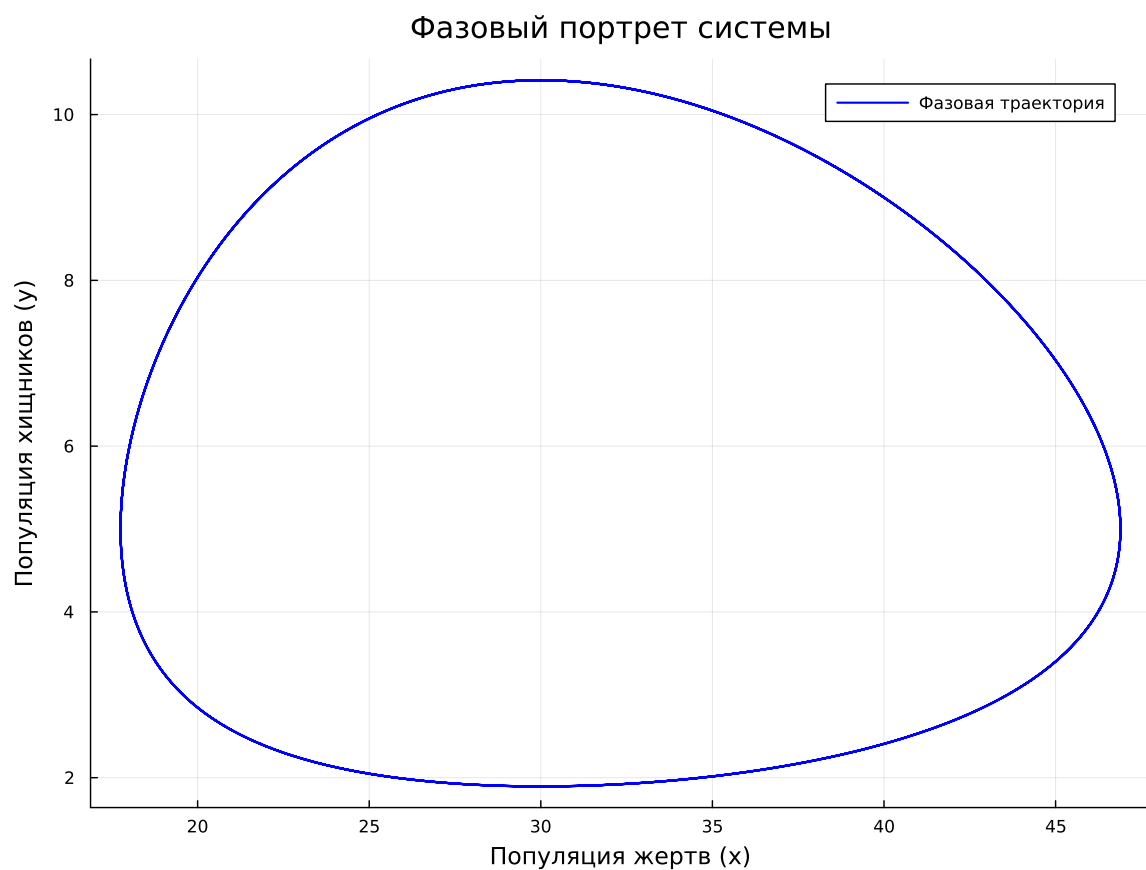


График 2: Фазовый портрет (хищники vs жертвы)


```
plt2 = plot(df_lv.prey, df_lv.predator,
            label="Фазовая траектория",
            xlabel="Популяция жертв (x)",
            ylabel="Популяция хищников (y)",
            title="Фазовый портрет системы",
            color=:blue,
            linewidth=1.5,
            grid=true,
            size=(800, 600),
            legend=:topright)
```



Добавление стрелок направления на фазовом портрете

```

step = 50 # шаг для отображения стрелок
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]], [df_lv.predator[i], df_lv.predator[i+step]],
        arrow=:closed, color=:blue, alpha=0.3, label=false)
end

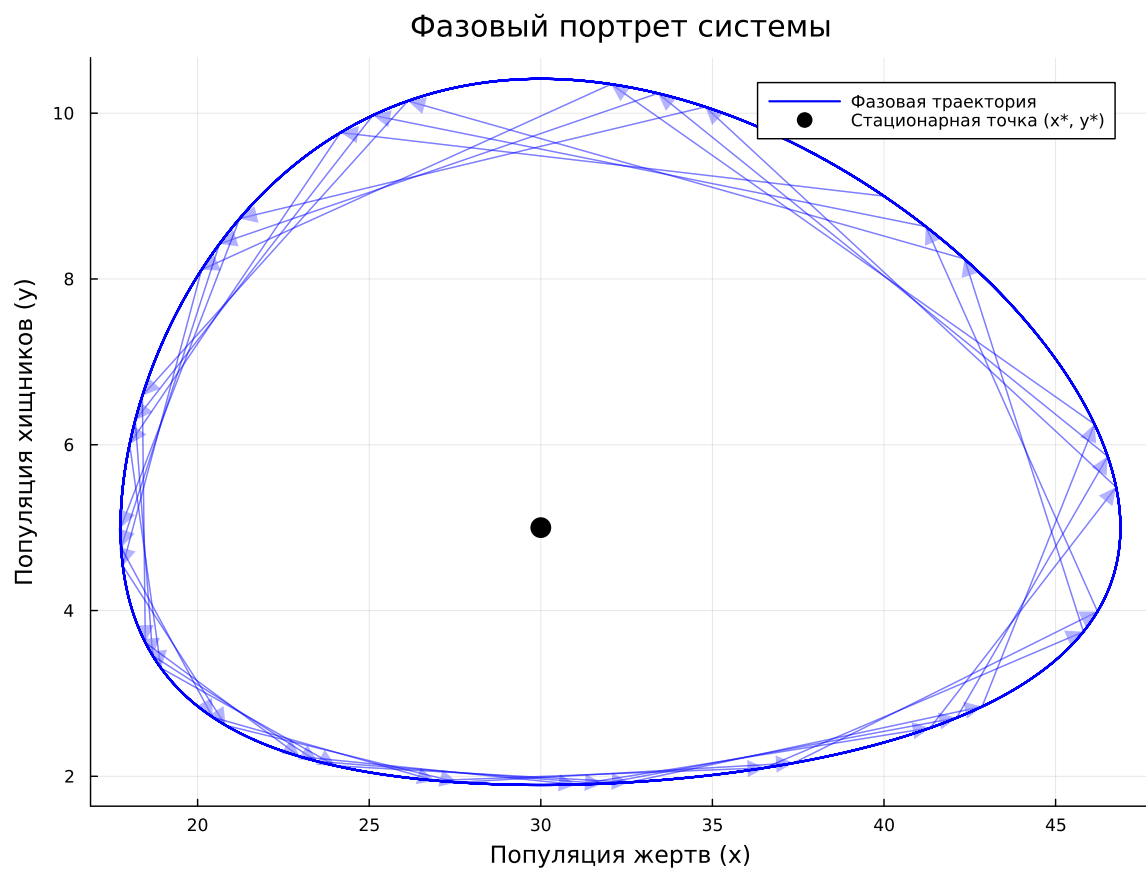
```

Добавление стационарной точки

```

scatter!(plt2, [x_star], [y_star], color=:black, markersize=8, label="Стационарная точка")

```



Изоклины (нулевого роста)

```

x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range) # y-изоклина (dy/dt = 0)
plot!(plt2, x_range, y_nullcline, color=:red, linestyle=:dash, linewidth=1.5, label="")

```

```

y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range))) # x-изоклина (dx/dt = 0)
plot!(plt2, x_nullcline, y_range, color=:green, linestyle=:dash, linewidth=1.5, label

```

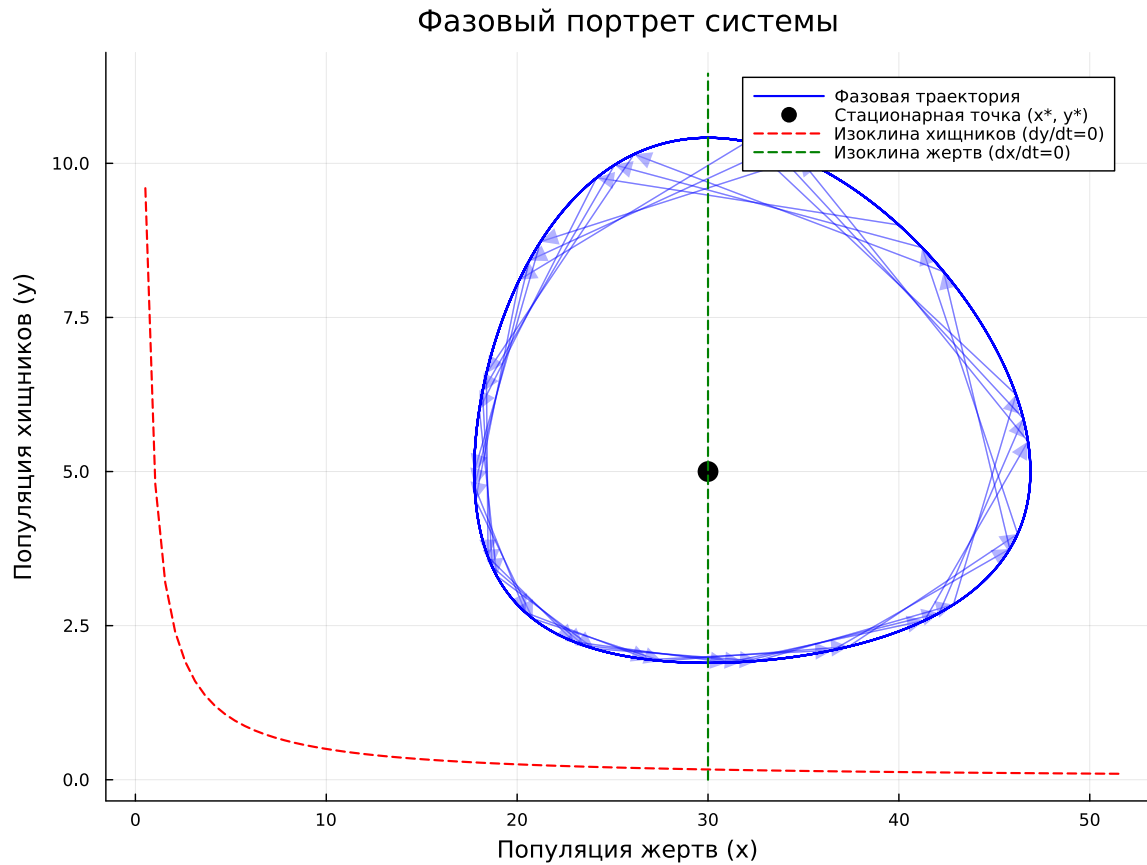


График 3: Производные (скорости изменения)

```

plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
  label=[L"dx/dt" L"dy/dt"],
  xlabel="Время",
  ylabel="Скорость изменения",
  title="Производные популяций",
  linewidth=1.5,
  legend=:topright,

```

```

grid=true,
size=(900, 400),
color=[:green :red])

```

```

hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=false)

```

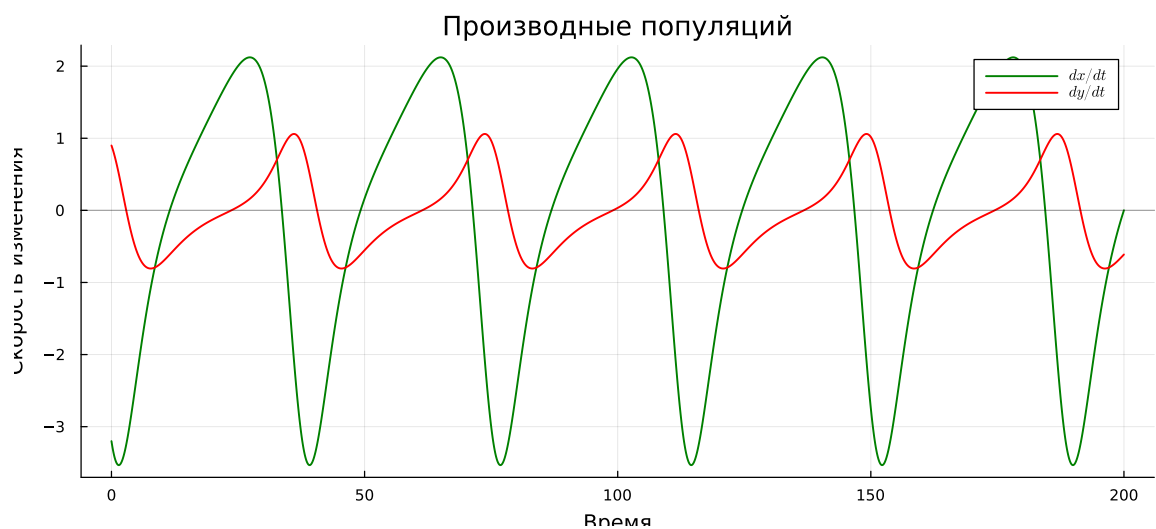


График 4: Относительные изменения (в %)

```

df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator .* 100

plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
    label=[L"dx/dt / x (%)" L"dy/dt / y (%)"],
    xlabel="Время",
    ylabel="Относительное изменение, %",
    title="Относительные темпы роста",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),

```

```
color=[:green :red])
```

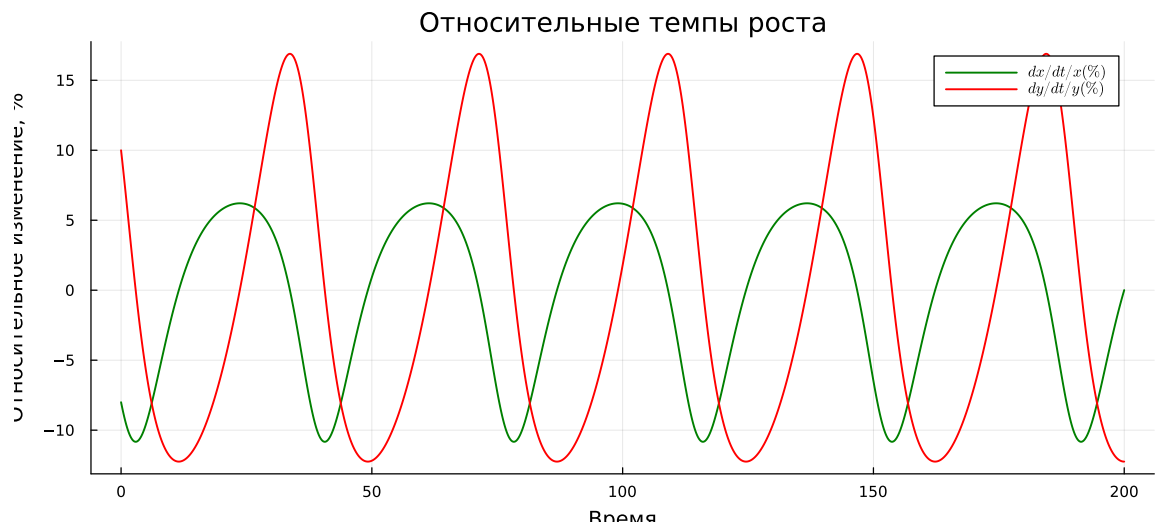


График 5: Спектральный анализ (быстрое преобразование Фурье)

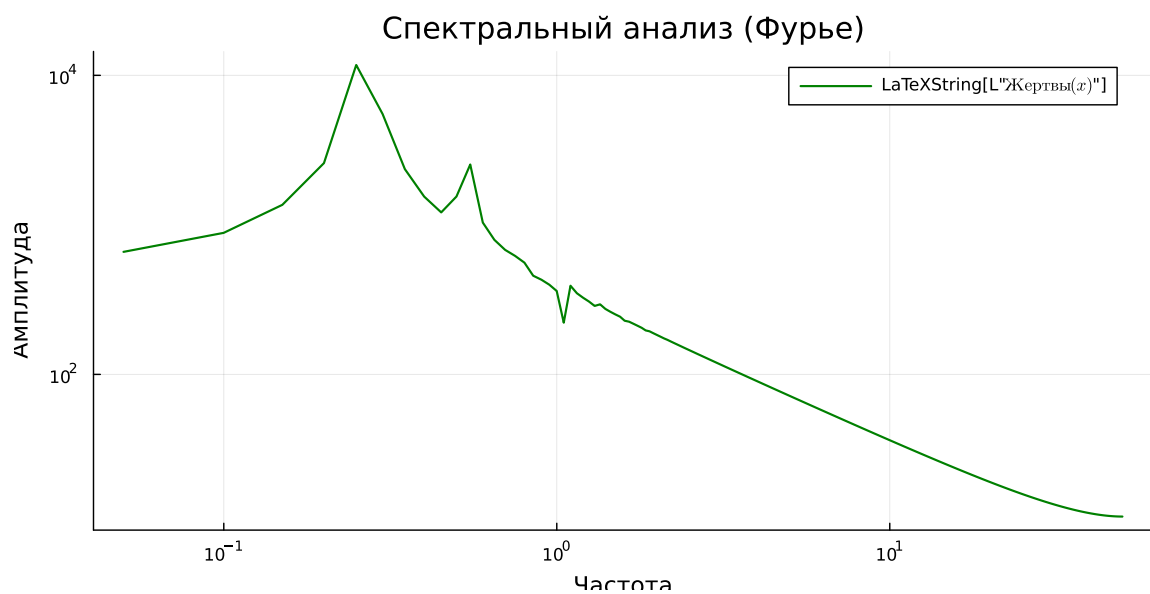
```
function compute_fft(signal, dt)
    n = length(signal)
    signal_clean = replace(signal, NaN => 0.0, Inf => 0.0, -Inf => 0.0)
    signal_centered = signal_clean .- mean(signal_clean)
    spectrum = abs.(rfft(signal_centered))
    freq = rfftfreq(n, 1/dt)
    positive_idx = (freq .> 0) .& (spectrum .> 0)
    return freq[positive_idx], spectrum[positive_idx]
end
```

compute_fft (generic function with 1 method)

Вычисление периодов колебаний

```
freq_pre, spectrum_pre = compute_fft(df_lv.prey, dt_lv)
freq_pred, spectrum_pred = compute_fft(df_lv.predator, dt_lv)
```

```
plt5 = plot(freq_prey, spectrum_prey,
            label=[L"Жертвы (x)"],
            xlabel="Частота",
            ylabel="Амплитуда",
            title="Спектральный анализ (Фурье)",
            linewidth=1.5,
            xscale=:log10,
            yscale=:log10,
            legend=:topright,
            grid=true,
            size=(800, 400),
            color=:green)
```

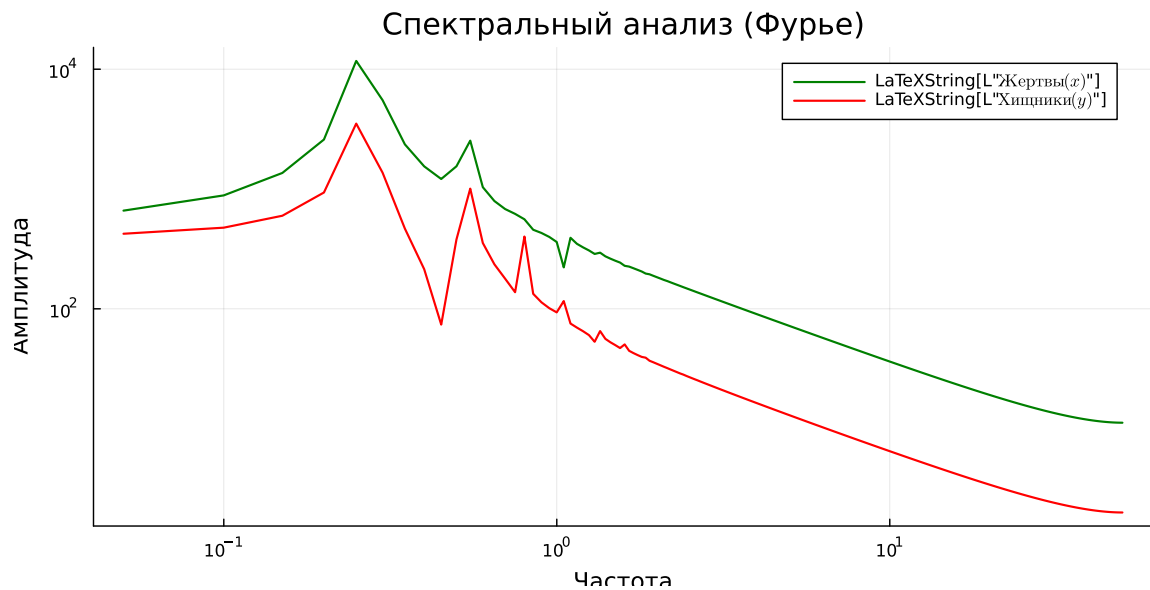


Добавляем спектр хищников, если данные корректны

```
if !isempty(freq_predator) && !isempty(spectrum_predator)
    plot!(plt5, freq_predator, spectrum_predator,
          label=[L"Хищники (y)"],
          color=:red,
```

```
linewidth=1.5)
```

```
end
```



Нахождение доминирующих частот

```
if !isempty(spectrum_pre) && length(spectrum_pre) > 1 # Пропускаем нулевую частоту
    idx_pre = argmax(spectrum_pre[2:end]) + 1
    if idx_pre <= length(freq_pre)
        dominant_freq_pre = freq_pre[idx_pre]
        period_pre = 1/dominant_freq_pre
        println("\nДоминирующая частота колебаний жертв: ", round(dominant_freq_pre,
        println("Период колебаний жертв: ", round(period_pre, digits=2), " единиц вр
    end
end
```

Доминирующая частота колебаний жертв: 0.2499 Гц

Период колебаний жертв: 4.0 единиц времени

График 6: Компактная панель всех графиков

```

plt6 = plot(layout=(3, 2), size=(1200, 900))

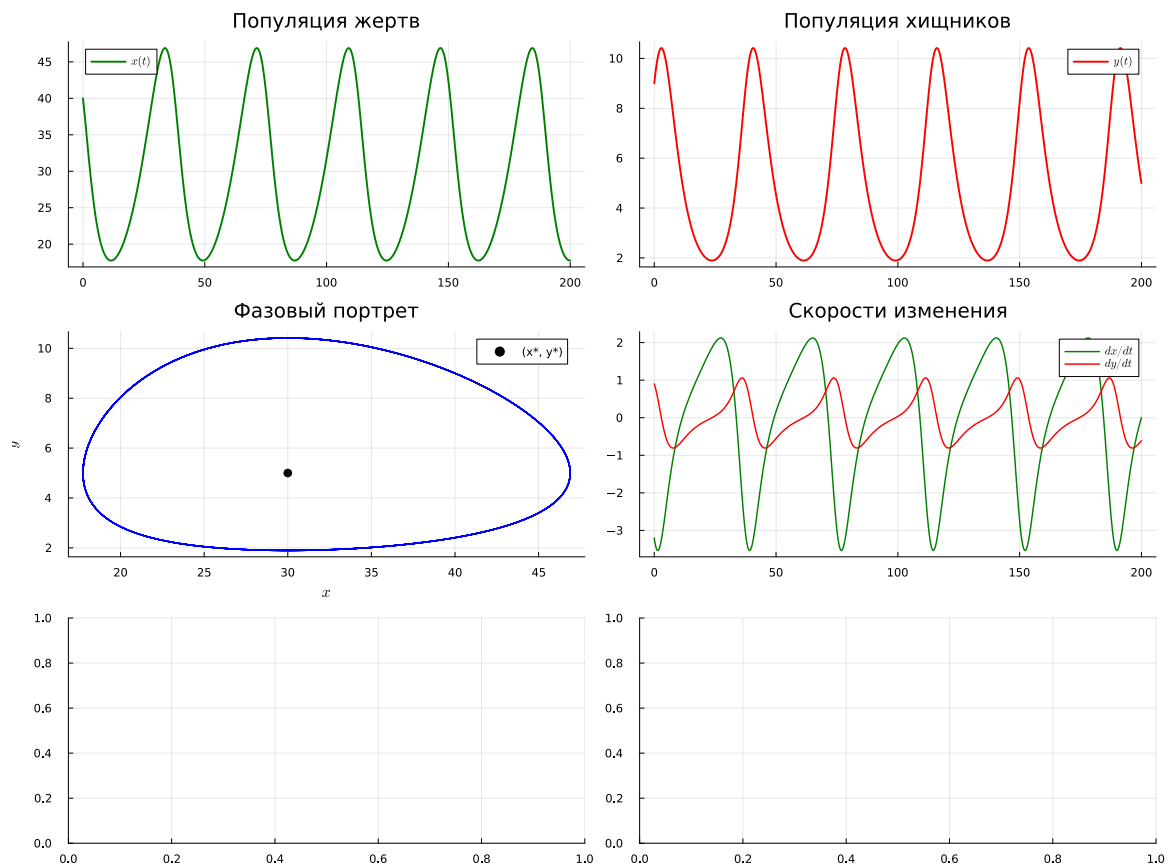
plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green, linewidth=2,
      title="Популяция жертв", grid=true)

plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red, linewidth=2,
      title="Популяция хищников", grid=true)

plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue, linewidth=1.5,
      title="Фазовый портрет", xlabel=L"x", ylabel=L"y", grid=true)
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, label="(x*, y*)")

plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
      label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
      title="Скорости изменения", grid=true, legend=:topright)

```

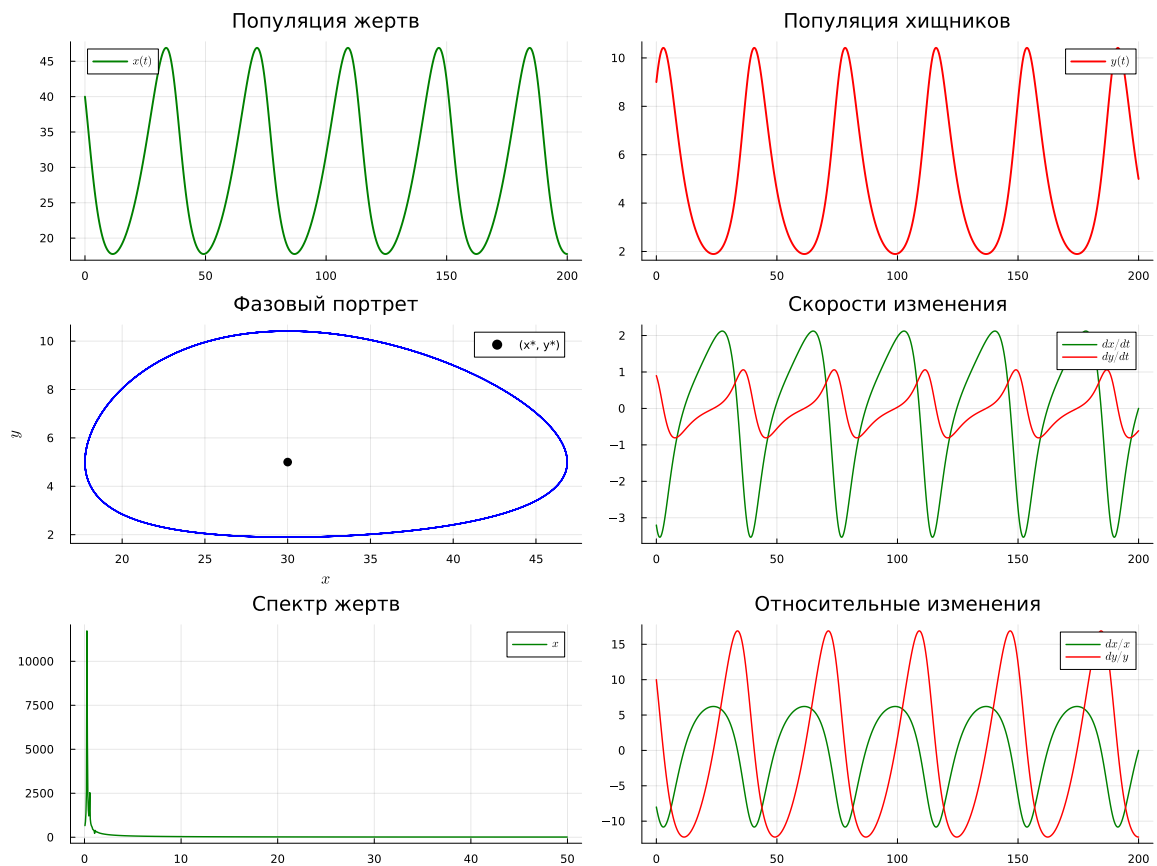
Для спектров в панели используем те же данные, но с линейной шкалой, чтобы избежать предупреждений

```

if !isempty(freq_pre) && !isempty(spectrum_pre)
    plot!(plt6[5], freq_pre, spectrum_pre, label=L"x", color=:green, linewidth=1.5,
        title="Спектр жертв", grid=true)
end

plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
    label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
    title="Относительные изменения", grid=true, legend=:topright)

```



Анализ результатов

```
println("\n" * "="^60)
println("Анализ результатов")
println("="^60)
println("\nОсновные статистики:")
println("Жертвы: min = ", round(minimum(df_lv.prey), digits=2),
      ", max = ", round(maximum(df_lv.prey), digits=2),
      ", mean = ", round(mean(df_lv.prey), digits=2))
println("Хищники: min = ", round(minimum(df_lv.predator), digits=2),
      ", max = ", round(maximum(df_lv.predator), digits=2),
      ", mean = ", round(mean(df_lv.predator), digits=2))
```

=====

Анализ результатов

=====

Основные статистики:

Жертвы: min = 17.75, max = 46.89, mean = 29.71

Хищники: min = 1.9, max = 10.41, mean = 5.2

Исправление 6: Упрощенный анализ колебаний с защитой от ошибок

```
function find_first_peak(signal, time)
    for i in 2:length(signal)-1
        if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

peak_time_pre, peak_value_pre = find_first_peak(df_lv.pre, df_lv.t)
peak_time_pred, peak_value_pred = find_first_peak(df_lv.predator, df_lv.t)

if !isnan(peak_time_pre) && !isnan(peak_time_pred)
    phase_shift = peak_time_pred - peak_time_pre
    println("\nАнализ колебаний:")
    println("Первый пик жертв: время = ", round(peak_time_pre, digits=2),
        ", значение = ", round(peak_value_pre, digits=2))
    println("Первый пик хищников: время = ", round(peak_time_pred, digits=2),
        ", значение = ", round(peak_value_pred, digits=2))
end
```

```
println("Сдвиг фаз (хищники отстают): ", round(phase_shift, digits=2))
end
```

Анализ колебаний:

Первый пик жертв: время = 33.7, значение = 46.89

Первый пик хищников: время = 2.9, значение = 10.41

Сдвиг фаз (хищники отстают): -30.8

Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
```

"/home/eavernikovskaya/work/study/2025-2026/Имитационное моделирование/simulation-modeling/labs/lab01/project/plots/lv_panel.png"

Дополнительный анализ: чувствительность к параметрам

```
println("\n\n" * "="^60)
println("Анализ чувствительности")
println("="^60)
```

```
=====
Анализ чувствительности
=====
```

Функция для анализа влияния параметров

```
function analyze_parameter_sensitivity(param_index, values, param_name)
  println("\nАнализ чувствительности к параметру: ", param_name)
  results = []
  for val in values
    p_test = copy(p_lv)
    p_test[param_index] = val
    prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_test)
    sol_test = solve(prob_test, Tsit5(), saveat=0.1)
    prey_end = sol_test.u[end][1]
    predator_end = sol_test.u[end][2]
    push!(results, (param=val, prey=prey_end, predator=predator_end))
    println(" $(param_name)=$(val): жертвы=$(round(prey_end,2)), хищники=$(round(
  end
  return results
end
```

analyze_parameter_sensitivity (generic function with 1 method)

Анализ чувствительности к ключевым параметрам

```
if false # Установите в true для выполнения анализа
  println("\n1. Влияние скорости размножения жертв (□):")
  analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "□")
  println("\n2. Влияние смертности хищников (□):")
  analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "□")
end

println("\nМоделирование завершено успешно!")
```

Моделирование завершено успешно!

2 Выводы

В ходе выполнения лабораторной работы №1 мы подготовили рабочее пространство для выполнения программ и приобрели необходимые навыки создания и преобразования программ на Julia

3 Список литературы

1. Лабораторная работа №1